

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

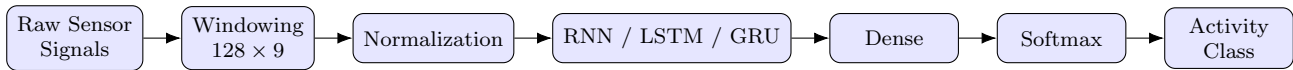
The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

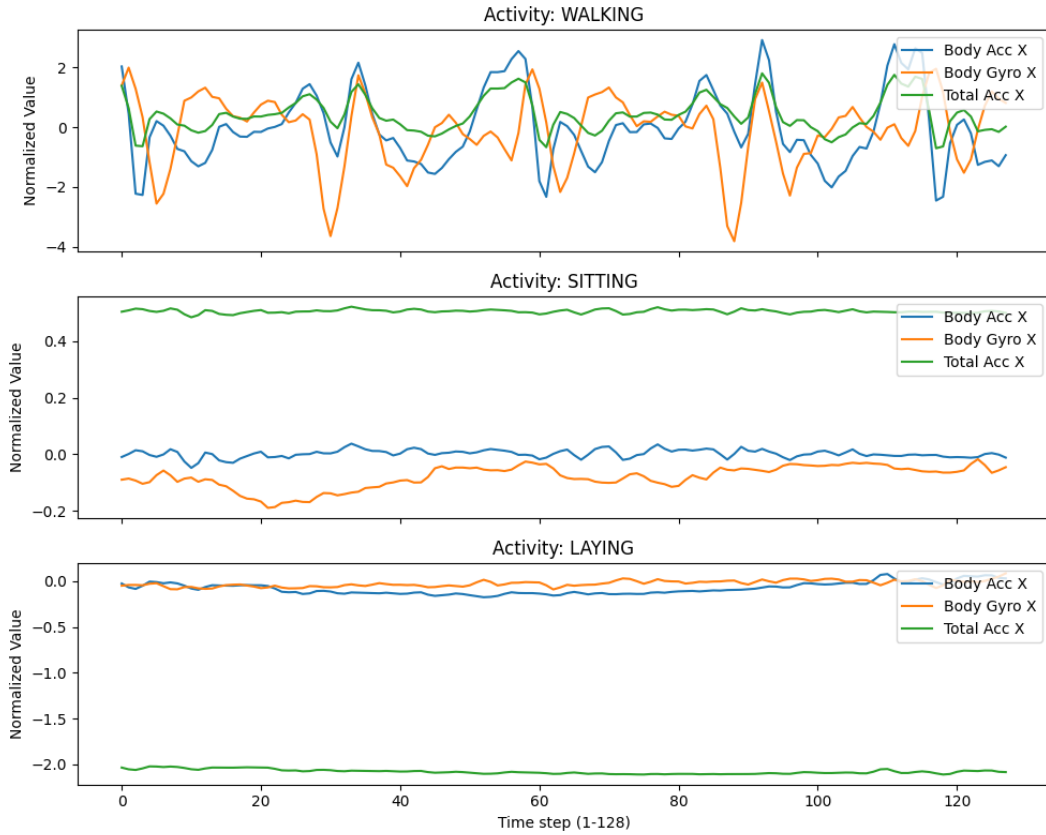


Figure 1: Sensor signal versus time for selected activities.

1. **What temporal pattern is visible?** Repetitive cyclic patterns for dynamic activities like walking, and steady flat patterns for static activities like sitting and laying.
2. **Which activities exhibit similar patterns?** Sitting and laying exhibit similar steady, low-variance patterns because both are stationary activities.
3. **Why is the ordering of measurements important?** The meaning of the data lies in how it changes over time; shuffling the sequence destroys the periodic motion patterns that characterize activities like walking.

7. Vanilla RNN Architecture

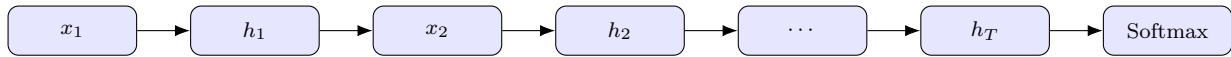
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

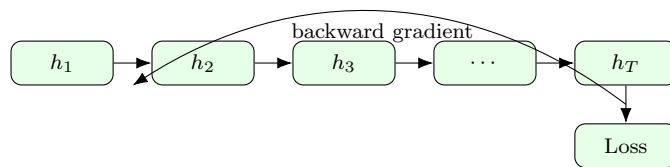
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

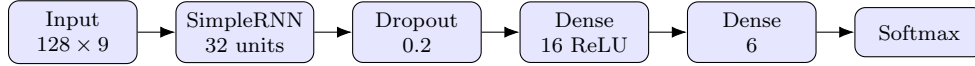
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

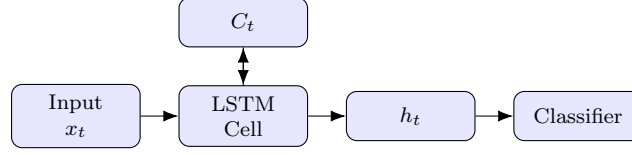
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Inference: The cell state acts as an internal memory that flows down the sequence with minimal linear interactions, which helps prevent gradients from vanishing. The gates (forget, input, output) dynamically decide what past information is no longer relevant (forget gate), what new information is important to add (input gate), and what part of the state should be exposed to the next layer (output gate).

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

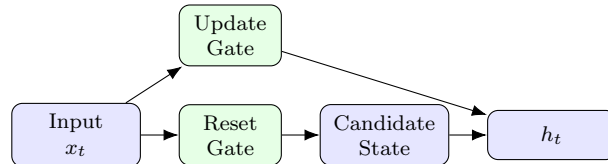
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Comparison: The Vanilla RNN has 0 gates and only maintains a single hidden state h_t . The LSTM introduces 3 gates (forget, input, output) and maintains two state representations: the cell state C_t (long-term memory) and the hidden state h_t (short-term memory). The GRU simplifies the LSTM by reducing the architecture to 2 gates (reset, update) and merges the cell state and hidden state into a single representation h_t .

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Training and Validation Curves plotted for RNN, LSTM and GRU

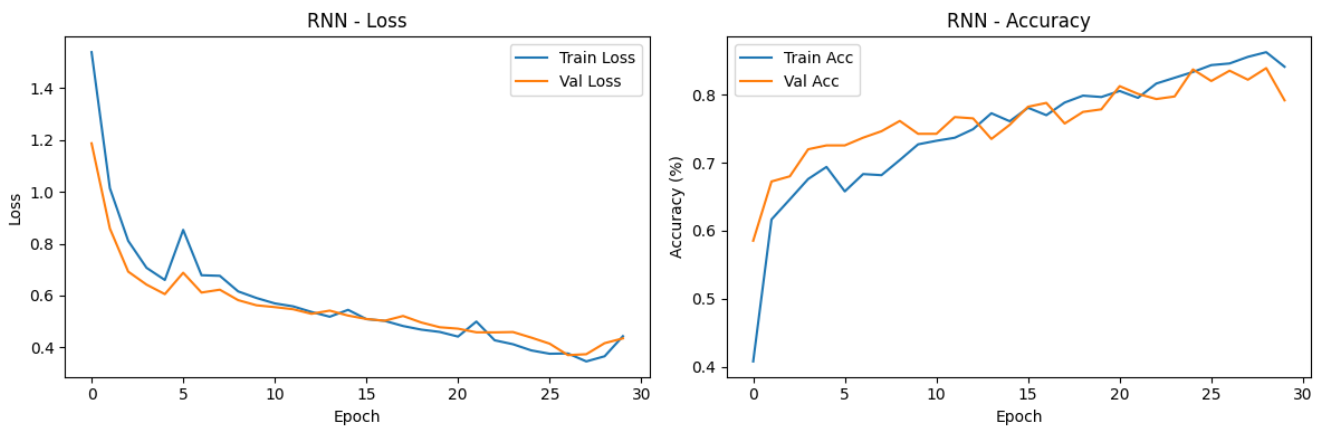


Figure 2: Training and Validation curves for Vanilla RNN.

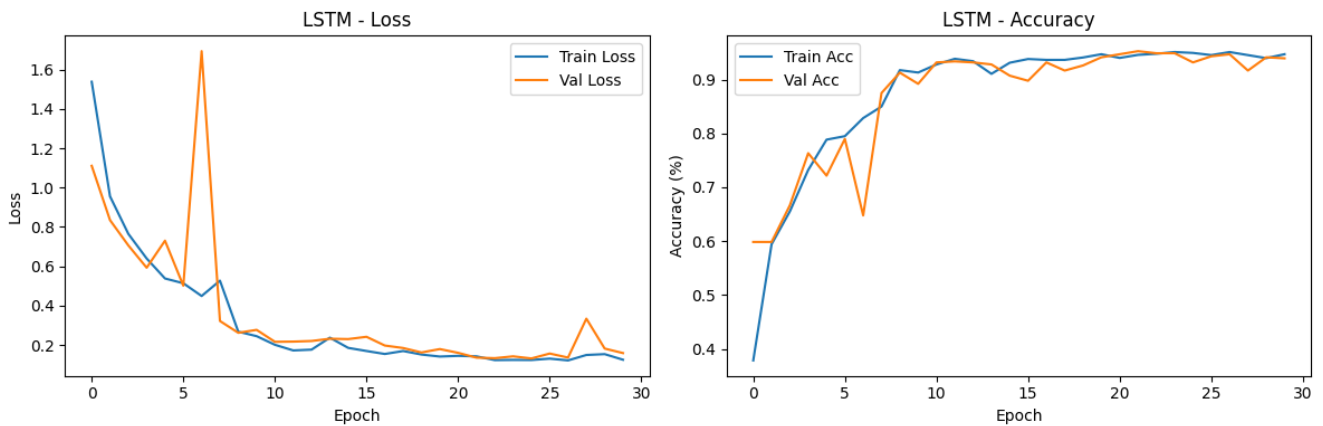


Figure 3: Training and Validation curves for LSTM.

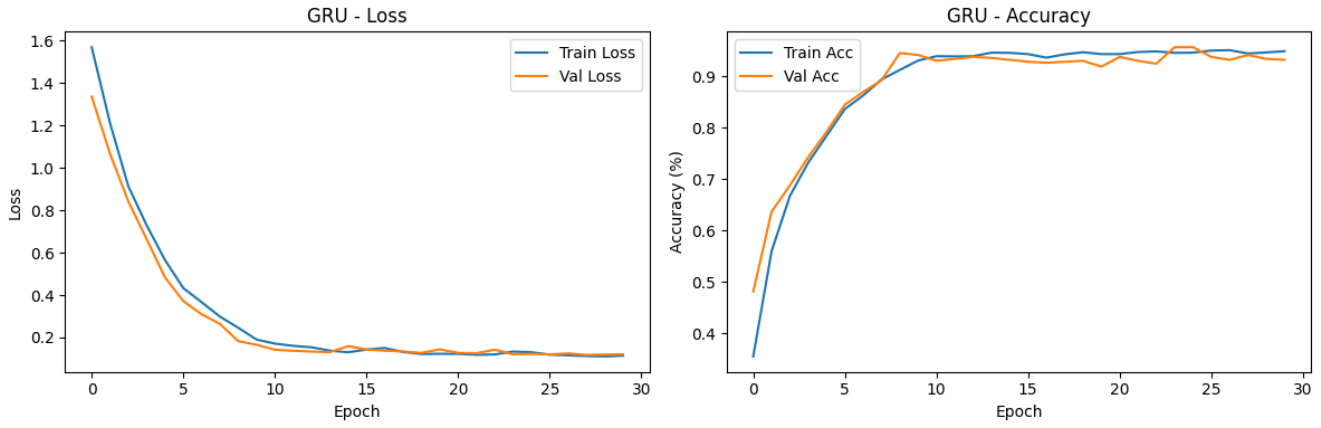


Figure 4: Training and Validation curves for GRU.

- **RNN:** There is a gradual decrease in training and validation loss, which is also oscillating, and the accuracy is stagnant at 76%. This indicates slow convergence and underfitting owing to the incapability of capturing the long-term dependency across 128 time-steps.
- **LSTM:** There is smooth and fast loss decreasing along with $\sim 90\%$ accuracy. There is stable and very fast convergence of the model than the RNN.
- **GRU:** It converges fast and smooth like LSTM, achieving $\sim 89\%$ accuracy. There is slightly fast convergence than LSTM, as GRU has fewer parameters.
- **Overfitting/Underfitting:** RNN underfits the data. LSTM and GRU overfit slightly at the end epochs, where the validation loss starts to level off or increase whereas the training loss keeps on decreasing.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	76.31	89.79	88.90
Macro Precision (%)	77.29	90.03	88.98
Macro Recall (%)	75.73	89.67	89.05
Macro F1 (%)	75.55	89.72	88.95
Parameters	1974	6006	4758
Training Time (s)	34.27	31.82	28.75

15. Confusion Matrix Analysis

Plot 5: Confusion Matrix for RNN, LSTM and GRU

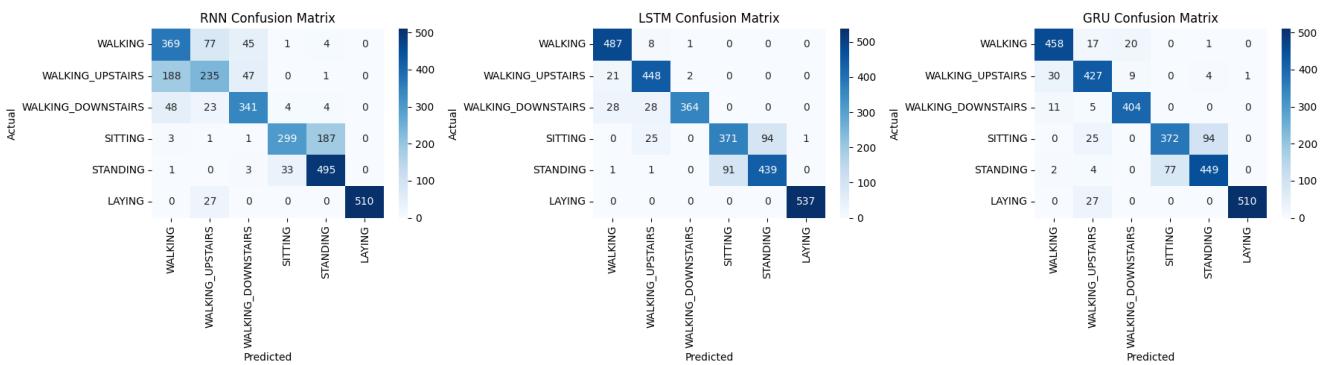


Figure 5: Confusion Matrices for RNN, LSTM, and GRU.

- **Highest recognition rate:** 'LAYING' is almost perfectly recognized across all models.
- **Largest misclassifications & Frequent confusion pairs:** 'SITTING' and 'STANDING' are the most miss-classified and confused actions amongst the five actions. This is so because both are static actions which have similar gravitational accelerations.
- **Consistency:** The errors are consistent across RNN, LSTM, and GRU, though RNN makes significantly more errors overall.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	34.27s	31.82s	28.75s
Test F1-score	75.55%	89.72%	88.95%

Plot 6: Model Performance Comparison

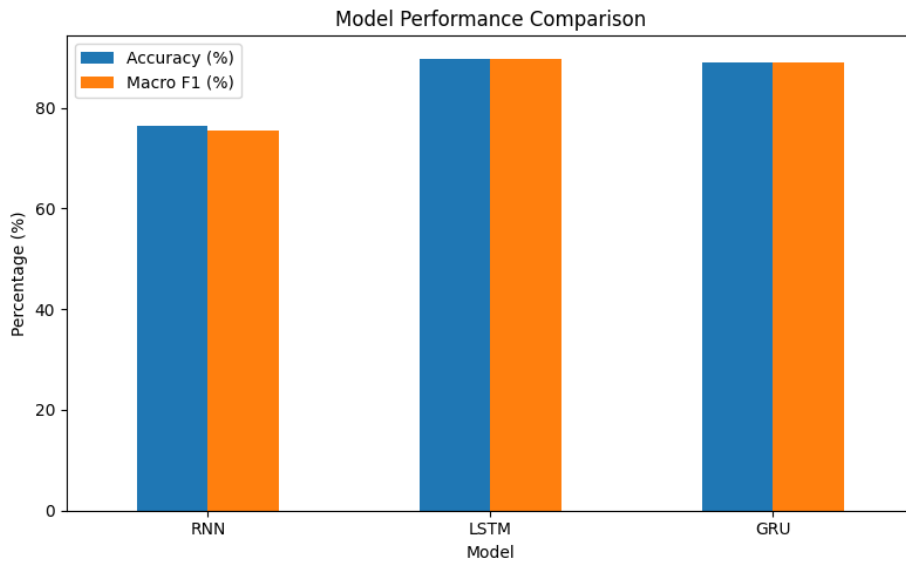


Figure 6: Comparison of Accuracy and Macro F1-score across models.

Inference: The RNN has the least complex model (1974 parameters), with the least cost of training and poor accuracy (76% Acc). The LSTM model is highly complex (6006 parameters) with high accuracy (90% Acc). The GRU model has a balanced complexity level (4758 parameters) and training time (28.75s) with high accuracy (89% Acc).

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	0.719	0.881	0.868
64	0.698	0.882	0.882
128	0.756	0.897	0.890

Plot 7: Sequence Length vs. Test F1-score

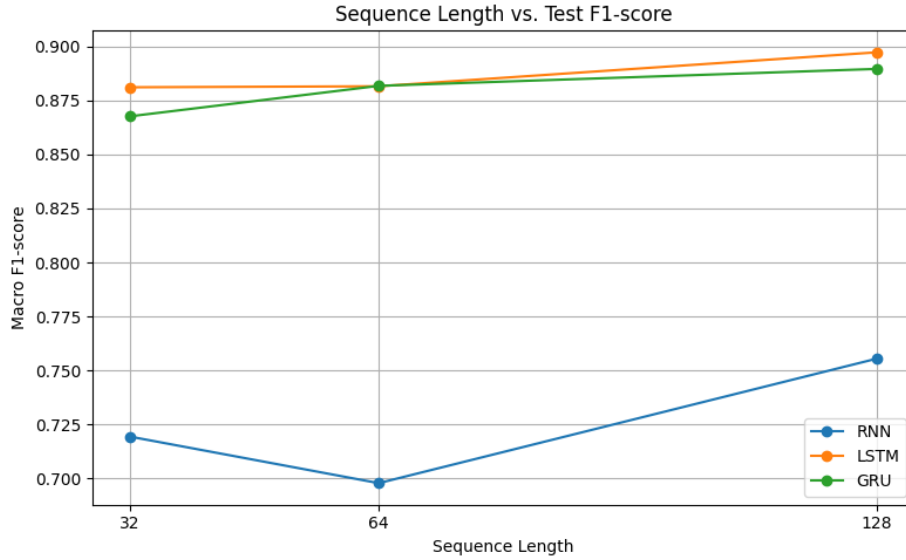


Figure 7: Effect of Sequence Length on Test Macro F1-score.

As the number of sequence length grows from 32 to 128 (for instance, F1-score of LSTM changes from 0.881 to 0.897), the F1 score also grows. The more context about time is available, the better classification can be done because complex actions like going upstairs require time to unfold their periodicity pattern.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Class 2 (Move Left)
Actual class : Class 2 (Move Left)
Confidence : 0.99

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



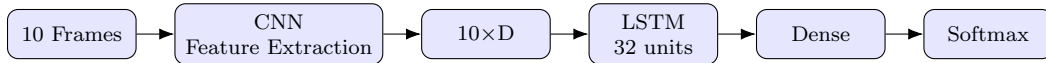
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Observation:

CNN feature dimensionality obtained via Global Average Pooling from MobileNetV2 is 1280. In case the video comprises of 10 frames, then the tensor dimensions that are fed into the recurrent network will be 10×1280 . For a batch comprising of B videos, the dimensions will be $B \times 10 \times 1280$.

21. CNN–LSTM Model

Use the following architecture:



Plot 8: Video Sample Frames

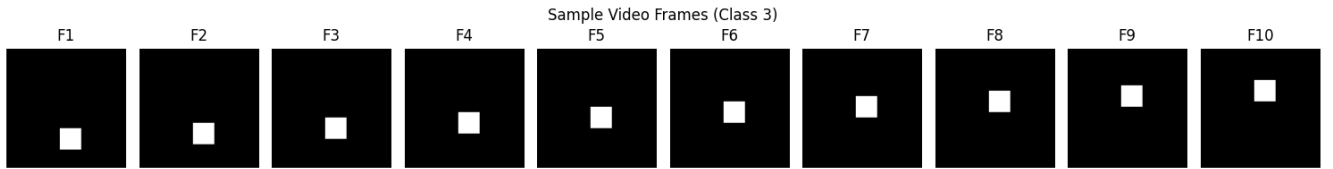


Figure 8: Sample frames from synthetic video dataset.

CNN processes the spatial data which includes the look, the form, and the location of objects within a frame. Recurrent Neural Network acquires temporal information by learning the movements, trajectory, and changes in the objects through the frames.

Plot 9: Video Training and Validation Curves

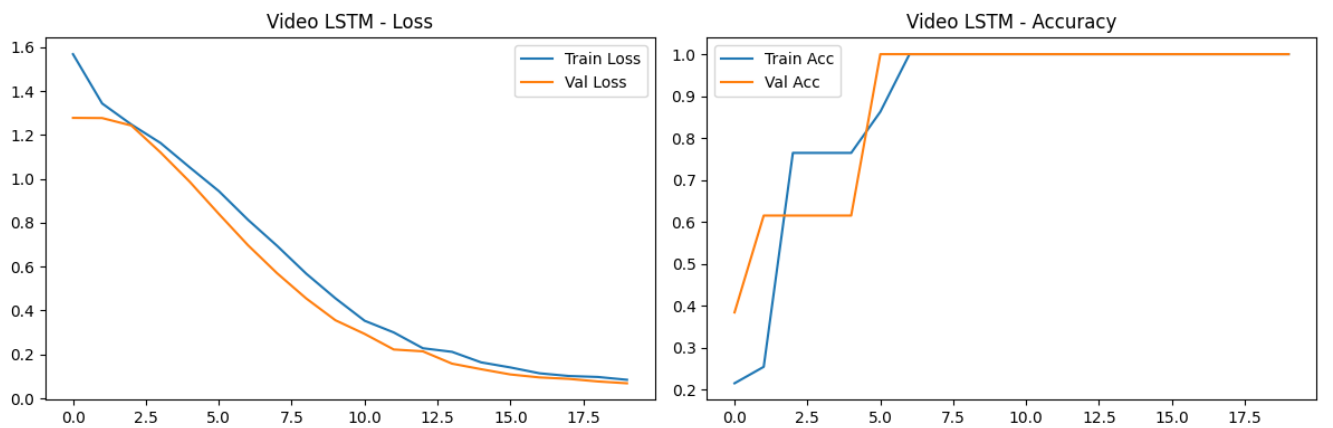


Figure 9: Training and validation curves for Video LSTM.

Plot 10: Video Confusion Matrix

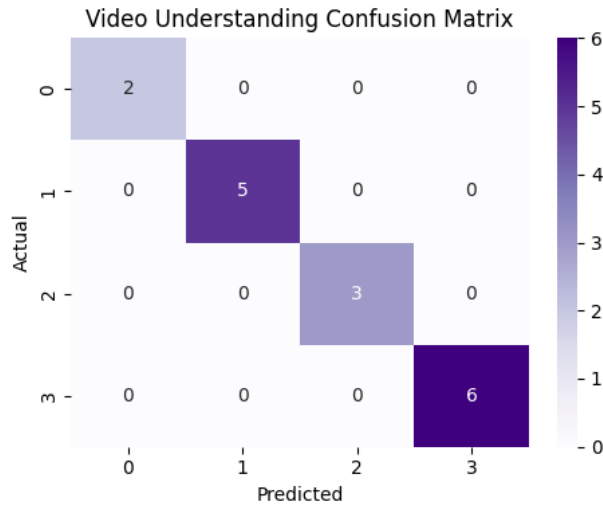


Figure 10: Confusion matrix for Video Understanding.

Inference: The recognition rate of classes is 100% accurate for the synthetic video data set. Inference of the direction of motion is accurately accomplished by the recurrent network. The reason for such a high level of accuracy is that the synthetic video data set contains only one object, which is a square moving in four different deterministic directions, without any movement of the camera and without any background.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

[5, 1, 8, 3].

Five test examples:

Sample	Input Sequence	Predicted Output
1	[5, 8, 9, 3]	[3, 9, 8, 5]
2	[1, 6, 8, 3]	[3, 8, 6, 1]
3	[7, 4, 4, 4]	[4, 4, 4, 7]
4	[8, 2, 3, 2]	[2, 3, 2, 8]
5	[6, 2, 3, 1]	[1, 3, 2, 6]

24. Sequence-to-Sequence Evaluation

Reporting:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Plot 11: Training and Validation curves for Sequence-to-Sequence learning

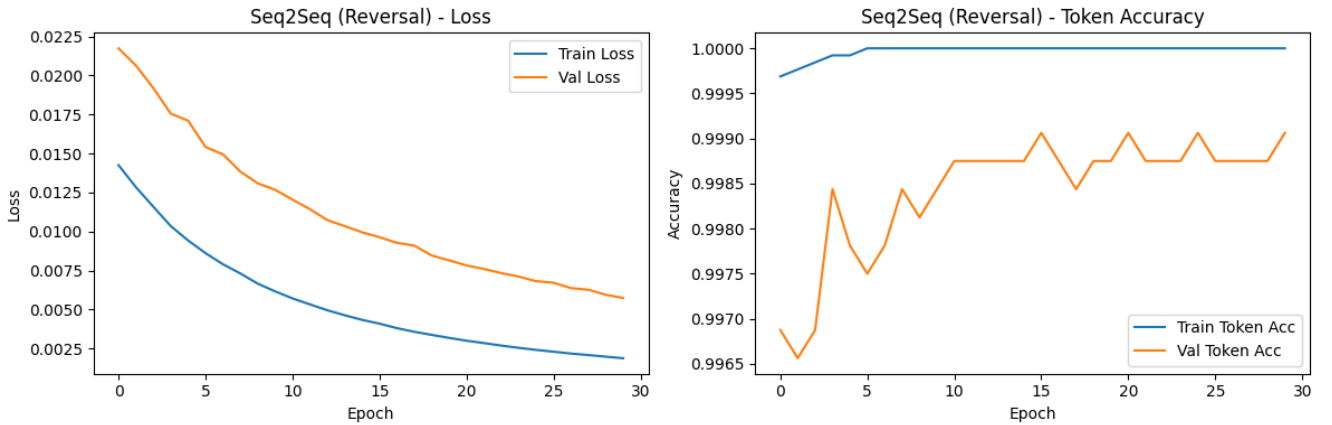


Figure 11: Training and Validation curves for Sequence-to-Sequence learning.

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Inference:

Token level accuracy is measured as the number of correct tokens (99.35%) whereas sequence accuracy is defined as the number of perfect sequence predictions (97.70%). The sequence level accuracy can be significantly lower than that of token accuracy since one wrong token means the whole sequence is wrong.

25. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	76.31%	77.29%	75.73%	75.55%	1974
LSTM	89.79%	90.03%	89.67%	89.72%	6006
GRU	88.90%	88.98%	89.05%	88.95%	4758
CNN-LSTM/GRU	100.00%	100.00%	100.00%	100.00%	~170000

Sequence-to-Sequence Model Results

Metric	Value
Token Accuracy	99.35%
Sequence Accuracy	97.70%

26. Discussion Questions

1. What is a recurrent neural network?

A recurrent neural network is a kind of neural network capable of processing data sequences with the help of a hidden state.

2. What information is represented by the hidden state?

The hidden state represents the memory of the network; that is, the context and information it has encountered so far in the sequence.

3. Why is sequence order important?

Sequence order is important because the meaning of the sequence data may depend on the time order of the data.

4. Explain the difference between a feed-forward network and an RNN.

A feed-forward network does not have recurrent connections and processes each input independently; an RNN processes each element of the sequence in a context with the previous elements.

5. Explain Backpropagation Through Time.

Backpropagation Through Time is the algorithm for optimizing the weights of an RNN during training, backpropagating the error from the last token to the beginning using dynamic unfolding of the RNN computational graph.

6. Why can gradients vanish during BPTT?

Gradients can vanish during BPTT because of the nature of the recurrent connections and the exponentiation of small values (resulting from the gradients of tanh in RNNs).

7. Why can gradients explode during BPTT?

Gradients can explode during BPTT because of the same nature of the recurrent connections and the exponentiation of big values.

8. What are long-term dependencies?

Long-term dependencies are a situation when information relevant to the prediction is located far away from the point of prediction in the sequence.

9. What is the role of the LSTM cell state?

The cell state in LSTM is a linear layer with some connections that carry information through the sequence with little modification; this prevents the gradient vanishing problem.

10. What are the forget, input and output gates in LSTM?

The forget gate learns when to forget information stored in the cell state; the input gate learns when to update the cell state with new information; the output gate learns when to write the hidden state from the cell state.

11. What are the reset and update gates in GRU?

The reset gate decides what information to forget about in the previous hidden state; the update gate learns how to combine the previous hidden state and the current candidate state into the current hidden state.

12. Compare LSTM and GRU.

LSTM features cell state and hidden state with three gates (forget, input, output), while GRU has fewer parameters since it collapses the cell state and the hidden state and uses two gates (reset, update).

13. Why does GRU generally use a simpler gating mechanism than LSTM?

GRU uses the simpler gating mechanism to reduce the number of parameters to optimize.

14. Why should RNN be compared with LSTM and GRU using the same experimental settings?

RNN, LSTM, and GRU should be compared using the same settings (e.g., data splits, optimizer, epochs), since different settings could affect the performance comparison between models.

15. What does the confusion matrix reveal that accuracy alone does not?

The confusion matrix reveals which classes are confused with which; in other words, it reflects the types of recognition errors, rather than a single accuracy value.

16. Why should macro F1-score be reported for multi-class classification?

The macro F1-score is reported because it computes the F1-score for each class independently, thus, not confusing classes with different sample distributions.

17. What is the significance of sequence length?

Sequence length determines how much context the model will be able to see, so that sufficient context is needed to identify patterns with a certain period.

18. Why can a CNN be used as a feature extractor for video?
CNN is the best architecture for visual tasks, so extracting spatial features for a single (2D) video frame is a matter for it.
19. What type of information is extracted by the CNN?
The CNN extracts spatial information about the video frames.
20. What type of information is learned by the recurrent network?
The recurrent network learns temporal information about the video.
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
They are arranged as a sequence because video is a temporal series of spatial information, so that the LSTM/GRU can learn its temporal structure.
22. Explain the encoder and decoder in the sequence-to-sequence learning.
The encoder reads an input sequence and encodes it into a hidden state; the decoder reads this hidden state and reconstructs the output sequence step-by-step.
23. What is teacher forcing?
Teacher forcing is a training technique where, instead of feeding the predicted outputs from the decoder at each time step, the true outputs are used at each time step (for training purposes).
24. Differentiate token accuracy and sequence accuracy.
Token accuracy is the percentage of tokens predicted correctly, while sequence accuracy is the percentage of predicted sequences that are completely correct (i.e., without any wrong tokens).
25. Why must the test set be untouched during model selection?
The test set should stay untouched to measure the generalization ability of the final model; using it during model selection will invalidate any comparisons due to data leakage.

27. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
For LSTM, 16 units gave 78.52% Accuracy, 2038 parameters, and took 9.6s. 64 units gave 87.75% Accuracy, 20086 parameters, and took 9.2s. Increasing units from 16 to 64 significantly improved accuracy at the cost of a $\sim 10\times$ increase in parameter count. The training time difference per epoch is small for small models on GPUs, but the complexity increase is evident in the parameter count.
2. Compare GRU and LSTM using the same number of recurrent units.
With 16 units: LSTM achieved 78.52% Acc (2038 Params); GRU achieved 68.04% Acc (1670 Params). With 64 units: LSTM achieved 87.75% Acc (20086 Params); GRU achieved 89.01% Acc (15542 Params). LSTM performed better at very low capacity (16 units), while GRU performed slightly better at higher capacity (64 units) despite having fewer parameters. GRU maintains a parameter efficiency advantage.

Plot 12: Effect of recurrent units on model performance across LSTM and GRU and 16 and 64 units

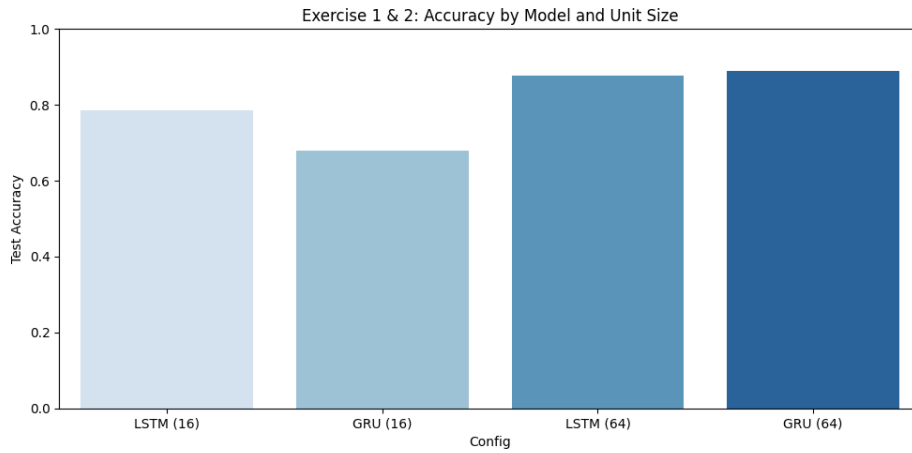


Figure 12: Effect of recurrent units on model performance.

3. Add a second recurrent layer and investigate whether performance improves.

The Stacked LSTM (two layers of 32 units) achieved an accuracy of 88.87%. Compared to the single-layer LSTM (89.78%), the performance slightly decreased. This suggests that for this specific dataset and sequence length, a deeper recurrent network might be harder to train or is prone to overfitting, and a single layer is sufficient.

4. Compare a bidirectional LSTM with a unidirectional LSTM.

The Bidirectional LSTM (32 units) achieved an accuracy of 89.72%, which is virtually identical to the unidirectional LSTM (89.78%). While bidirectional models capture context from both directions, real-time physical activity recognition using sensor windows doesn't necessarily benefit from future context as much as tasks like language translation do.

Plot 13: Comparison of different recurrent architectures (Stacked, Bidirectional)

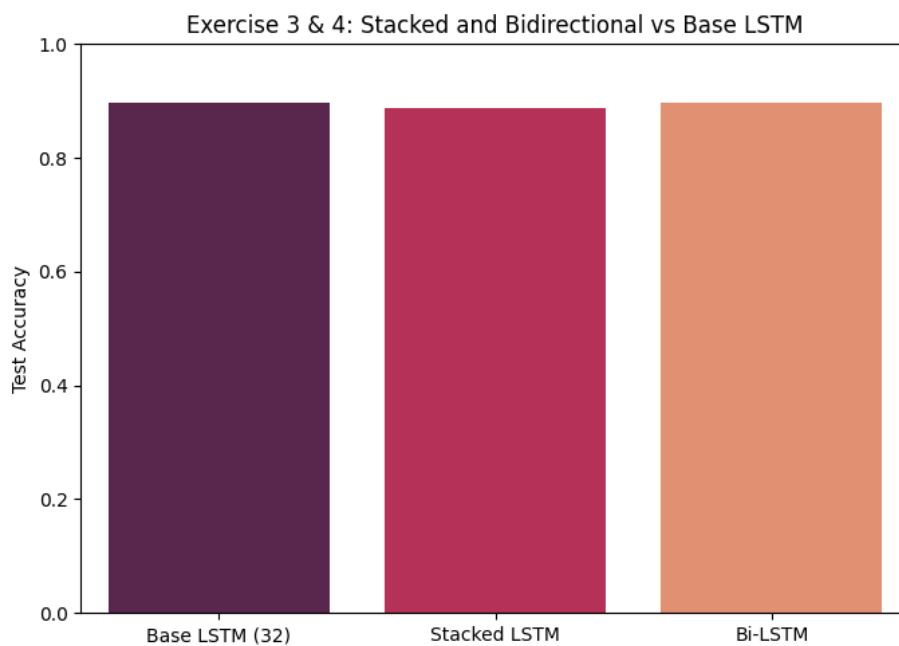


Figure 13: Comparison of different recurrent architectures (Stacked, Bidirectional).

5. Change the sequence length and study the effect on computational cost.

Training time for 10 epochs: Length 32 took 7.17s, Length 64 took 8.02s, Length 128 took 10.26s. Training time scales linearly with sequence length in BPTT. Longer sequences require more sequential unrolling operations and memory for hidden states, steadily increasing computational cost.

Plot 14: Effect of sequence length on computational cost

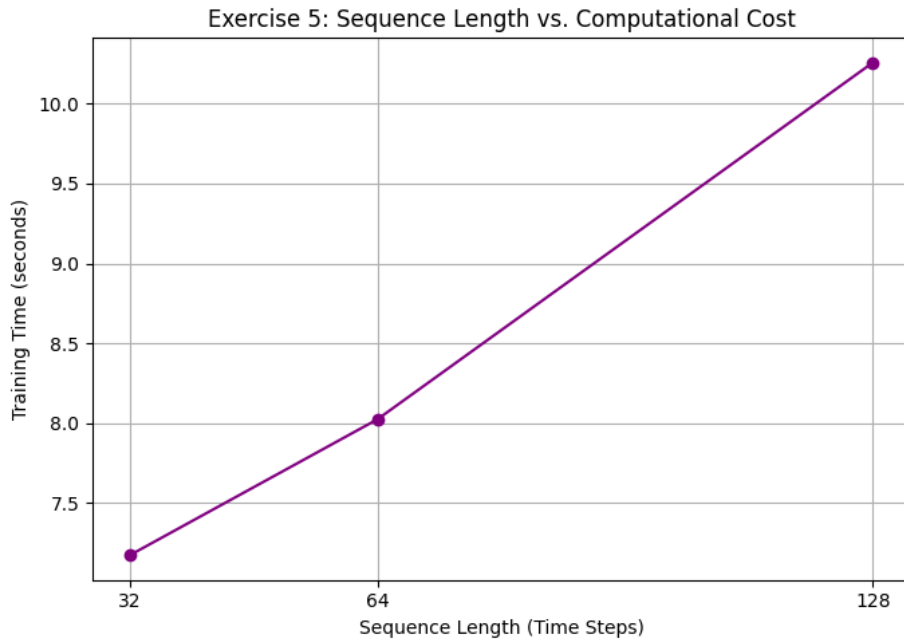


Figure 14: Effect of sequence length on computational cost.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

Video LSTM achieved 100% Accuracy in 4.29s. Video GRU achieved 100% Accuracy in 4.10s. Both models perfectly learned the synthetic video motion. GRU trained slightly faster than LSTM, confirming its computational efficiency when classifying short feature sequences.

Plot 15: Video task comparison between LSTM and GRU

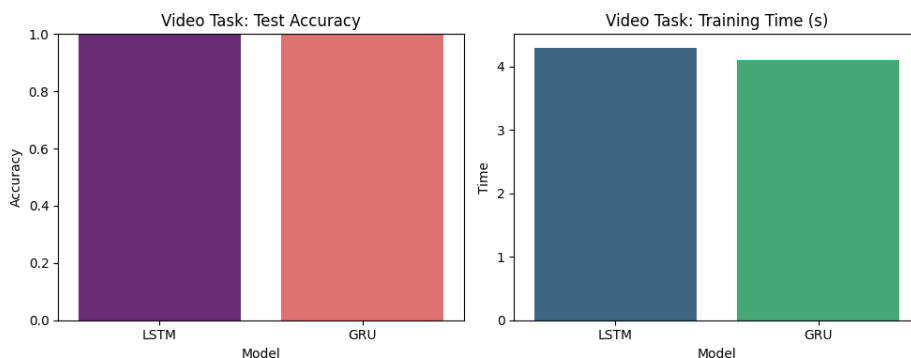


Figure 15: Video task comparison between LSTM and GRU.

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

The seq2seq architecture was successfully modified by changing the `RepeatVector` to length 5 (input length was 4) and configuring the decoder's dense layer to output the padded sequence.

The Encoder-Decoder structure naturally handles mismatched sequence lengths because the encoder compresses the input into a fixed-size context vector, and the decoder can be unrolled for an arbitrary number of steps independent of the input length.

14. Source Code

[GitHub Repository](#)

29. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>